Формальные грамматики

Лекция 4

Формальное определение языка. Понятие языка

- **Язык** – это заданный набор символов и правил, устанавливающих способы комбинации этих символов между собой для записи осмысленных предложений.
- **Алфавит** – набор допустимых символов языка. Алфавит – счетное, непустое множество символов.
- Цепочка символов α является цепочкой над алфавитом $\alpha(V)$, если в нее входят только символы, принадлежащие алфавиту V.
- Для любого алфавита V пустая цепочка ε может как являться, так и не являться цепочкой над этим алфавитом.
- Если $|V|=0$ и V – множество, то оно называется пустым множеством и обозначается $.

$$|\varepsilon|=0$$

$$|\{\varepsilon\}|=1$$

Формальное определение языка. Понятие языка

- V* множество, содержащее все цепочки в алфавите V, включая пустую цепочку ε.
- V* - **итерация** множества V или транзитивное замыкание.
- V+ - множество всех цепочек длиной 1 и более, исключив тем самым цепочку ε.
- V+ - **усечённая итерация** множества V или усеченное транзитивное замыкание.

  $V^* = V^+ \cup \{\varepsilon\}$

  V= {a,b,c}

  V* = {a, b, c, aa, bb, cc, aab, abc, abbc … ε}

  V+ = {a, b, c, aa, bb, cc, aab, abc, abbc …}
- **Декартовым произведением** $A \times B$ множеств A и B называется множество $\{ \alpha\beta \mid \alpha \in A, \beta \in B\}$.

  Если A= {a,b} и B={c,d} , то $A \times B$ = {ac, ad, bc, bd}

Формальное определение языка. Понятие языка

- **Языком L** над алфавитом V называют некоторое счетное подмножество цепочек конечной длины из множества всех цепочек над алфавитом V.
  $L(V) \leq V^*$
  - ✓ множество цепочек языка не обязано быть конечным
  - ✓ хотя каждая цепочка в языке обязана быть конечной длины, эта длина формально ничем не ограничена
- **Предложением языка** называется цепочка символов, принадлежащая этому языку.

Формальное определение языка. Понятие языка

- Язык L над алфавитом V включает в себя язык L′ над алфавитом V ($L(V) \leq L'(V)$), если справедливо, что любая цепочка α принадлежащая L, принадлежит и L′.
  $\alpha \in L(V)$ и $\alpha \in L'(V)$
- **Два языка** L(V) и L′ (V) **равны** или совпадают если справедливо $L(V) \leq L'(V)$ и $L'(V) \leq L(V)$.
- **Два языка** L(V) и L’(V) **почти эквиваленты**, если они отличаются на пустую цепочку $L(V) =\sim L'(V)$.

$$L(V) \cup \{\varepsilon\} = L'(V) \cup \{\varepsilon\} .$$

Способы задания языка

- перечисление всех допустимых цепочек языка
- с помощью указания способа порождения цепочек языка (задать грамматику языка, используемую для порождения языка)

$L(G1)=\{0^n1^n \mid n>0\}$

01

0011

000111

$L(G1)=\{0^n1^m \mid n>0, m>0\}$

01

0000011

00111

- определение метода распознавания цепочек языка

Понятие о грамматике языка

- **Грамматика** – описание способов построения предложений некоторого языка.
- Грамматика — один из основных подходов к описанию бесконечного формального языка конечными средствами.
- **Правило** (**продукция**) – упорядоченная пара цепочек (α β), которое записывается $\alpha \rightarrow \beta$ (α порождает β).
- L(G) – язык L, заданный грамматикой G.

Понятие о грамматике языка

- Грамматики G1 и G2 называются **эквивалентными**, если L(G1) = L(G2).

  G1 = ({0,1}, {A,S}, P1, S)

  P1: $S \to 0A1$

  $0A \to 00A1$

  $A \to \varepsilon$

  G2 = ({0,1}, {S}, P2, S)

  P2: $S \to 0S1 \mid 01$

  $$L(G1) = L(G2) = \{0^n1^n \mid n>0\}.$$

- Грамматики G1 и G2 **почти эквивалентны**, если $L(G1) \cup \{\varepsilon\} = L(G2) \cup \{\varepsilon\}$.

  G1 = ({0,1}, {A,S}, P1, S)

  P1: $S \to 0A1$

  $0A \to 00A1$

  $A \to \varepsilon$

  G2 = ({0,1}, {S}, P2, S)

  P2: $S \to 0S1 \mid \varepsilon$

  $L(G1)=\{0^n1^n \mid n>0\}$

  $L(G2)=\{0^n1^n \mid n>=0\}$

Формальное определение грамматики

По определению Хомского формальная грамматика представляет собой четвёрку:

$$\mathbf{G=\{VT, VN, P, S\}}$$

- VT (T) - множество терминальных символов языка,
- VN (N) – множество нетерминальных символов (или понятий языка или синтаксических единиц)

  $V = VN \cup VT$

  $VN \cap VT = \varnothing$
- P – множество правил подстановки (продукций), имеющий вид $\alpha \rightarrow \beta$, $\alpha \in V^{+}$, $\beta \in V^{*}$.

  Знак $\rightarrow$ означает "непосредственно порождает "или "есть по определению".
- S – аксиома грамматики или начальный символ грамматики. $S \in VN$.

Пример формальной грамматики

Грамматика, определяющая целое число без знака:

$$G=\{VT, VN, P, S\}$$

$VN = \{A,B\}$

$VT = \{0,1,2,3,4,5,6,7,8,9\}$

$P = \{A \rightarrow BA,\ A \rightarrow B,\ B \rightarrow 0,\ B \rightarrow 1, \ldots\ B \rightarrow 9\}$

$S = \{A\}$

A - целое число без знака, B - любая цифра.

СПОСОБЫ ЗАПИСИ СИНТАКСИСА ЯЗЫКА

Метаязык - язык, предназначенный для описания другого языка

Метаязык Хомского

- → символ отделяет левую часть правила от правой (читается как "порождает" и "это есть");
- **нетерминалы** обозначаются буквой А с индексом, указывающим на его номер;
- **терминалы** - это символы, используемые в описываемом языке;
- Каждое **правило** определяет порождение одной новой цепочки, причём один и тот же нетерминал может встречаться в нескольких правилах слева.

Метаязык Хомского

1. $A_1 \to A$	17. $A_1 \to Q$	33. $A_1 \to g$	49. $A_1 \to w$
2. $A_1 \to B$	18. $A_1 \to R$	34. $A_1 \to h$	50. $A_1 \to x$
3. $A_1 \to C$	19. $A_1 \to S$	35. $A_1 \to i$	51. $A_1 \to y$
4. $A_1 \to D$	20. $A_1 \to T$	36. $A_1 \to j$	52. $A_1 \to z$
5. $A_1 \to E$	21. $A_1 \to U$	37. $A_1 \to k$	53. $A_2 \to 0$
6. $A_1 \to F$	22. $A_1 \to V$	38. $A_1 \to l$	54. $A_2 \to 1$
7. $A_1 \to G$	23. $A_1 \to W$	39. $A_1 \to m$	55. $A_2 \to 2$
8. $A_1 \to H$	24. $A_1 \to X$	40. $A_1 \to n$	56. $A_2 \to 3$
9. $A_1 \to I$	25. $A_1 \to Y$	41. $A_1 \to o$	57. $A_2 \to 4$
10. $A_1 \to J$	26. $A_1 \to Z$	42. $A_1 \to p$	58. $A_2 \to 5$
11. $A_1 \to K$	27. $A_1 \to a$	43. $A_1 \to q$	59. $A_2 \to 6$
12. $A_1 \to L$	28. $A_1 \to b$	44. $A_1 \to r$	60. $A_2 \to 7$
13. $A_1 \to M$	29. $A_1 \to c$	45. $A_1 \to s$	61. $A_2 \to 8$
14. $A_1 \to N$	30. $A_1 \to d$	46. $A_1 \to t$	62. $A_2 \to 9$
15. $A_1 \to O$	31. $A_1 \to e$	47. $A_1 \to u$	63. $A_3 \to A_1$
16. $A_1 \to P$	32. $A_1 \to f$	48. $A_1 \to v$	64. $A_3 \to A_3 A_1$
			65. $A_3 \to A_3 A_2$

Описание идентификатора на метаязыке Хомского

Бэкуса-Наура форма (БНФ)

- Запись правила разбита на две части разделенных символом определения **::=** или иногда вместо него используют →.
- В левой части содержится определяемый символ, а справа последовательность из букв и символов.
- В описании правила в правой части может использоваться оператор выбора “**|**” обеспечивающий логическое **ИЛИ**.
- В описании правила может применяться рекурсия.

Бэкуса-Наура формы (БНФ)

- символ "::=" отделяет левую часть правила от правой;
- **нетерминалы** обозначаются произвольной символьной строкой, заключённой в угловые скобки "<" и ">";
- **терминалы** – это символы, используемые в описываемом языке;
- каждое **правило** определяет порождение нескольких альтернативных цепочек, отделяемых друг от друга символом вертикальной черты “|”.

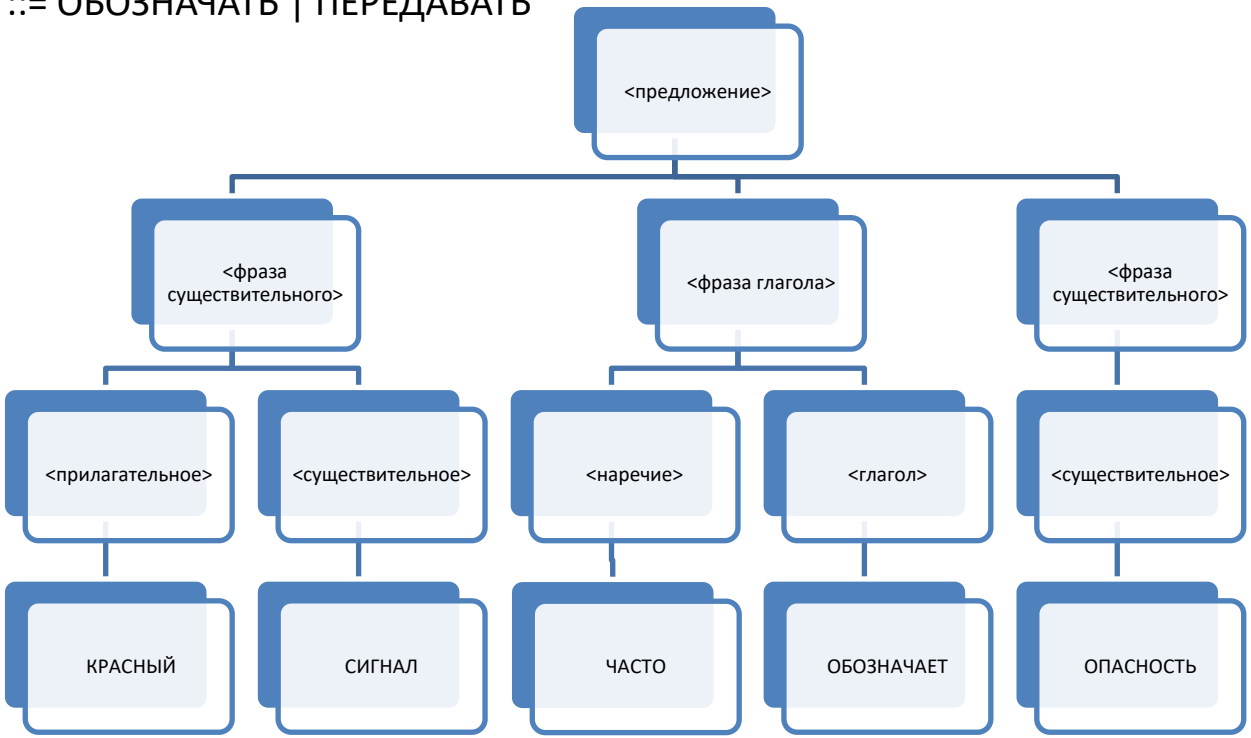
Бэкуса-Наура форма (БНФ)

1. <буква> ::= A | B| C …| Z| a| b| c| …| z
2. <цифра> ::= 0| 1| 2 … |9
3. <идентификатор> ::= <буква> |
   <идентификатор><буква>|
   <идентификатор><цифра>

Пример описания идентификатора с использованием БНФ

Бэкуса-Наура форма (БНФ)

- <предложение> ::= <фраза существительного> <фраза глагола> <фраза существительного>
- < фраза существительного > ::= <прилагательное> <существительное> | <существительное>
- <прилагательное> ::= КРАСНЫЙ| ЗЕЛЕНЫЙ
- <существительное> ::= СИГНАЛ| ФАКЕЛ| ОПАСНОСТЬ
- <фраза глагола> ::= <наречие> <глагол> | <глагол>
- < наречие> ::= ВСЕГДА | ИНОГДА| ЧАСТЬ| РЕДКО
- <глагол> ::= ОБОЗНАЧАТЬ | ПЕРЕДАВАТЬ



Упрощенная грамматика русского языка в терминах БНФ

РБНФ (расширенная)

- [ ] – синтаксическая конструкция может отсутствовать;
- { } – повторение синтаксической конструкции (возможно 0 раз)
- ( ) – для ограничения альтернативных конструкций
- {\ \} – для обозначения повторения один и более раз.

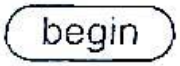
РБНФ

- <буква> ::= A | B| C …| Z| a| b| c| …| z
- <цифра> ::= 0| 1| 2 … |9
- <идентификатор> ::= <буква> {<буква> | <цифра>}

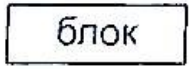
Пример описания идентификатора с использованием РБНФ

Диаграмма Вирта

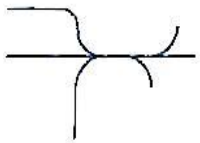
терминальный символ, принадлежащий алфавиту языка



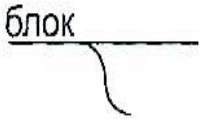
постоянная группа терминальных символов, определяющая название лексемы, ключевое слово и т.д.



нетерминальный символ определяющий название правила

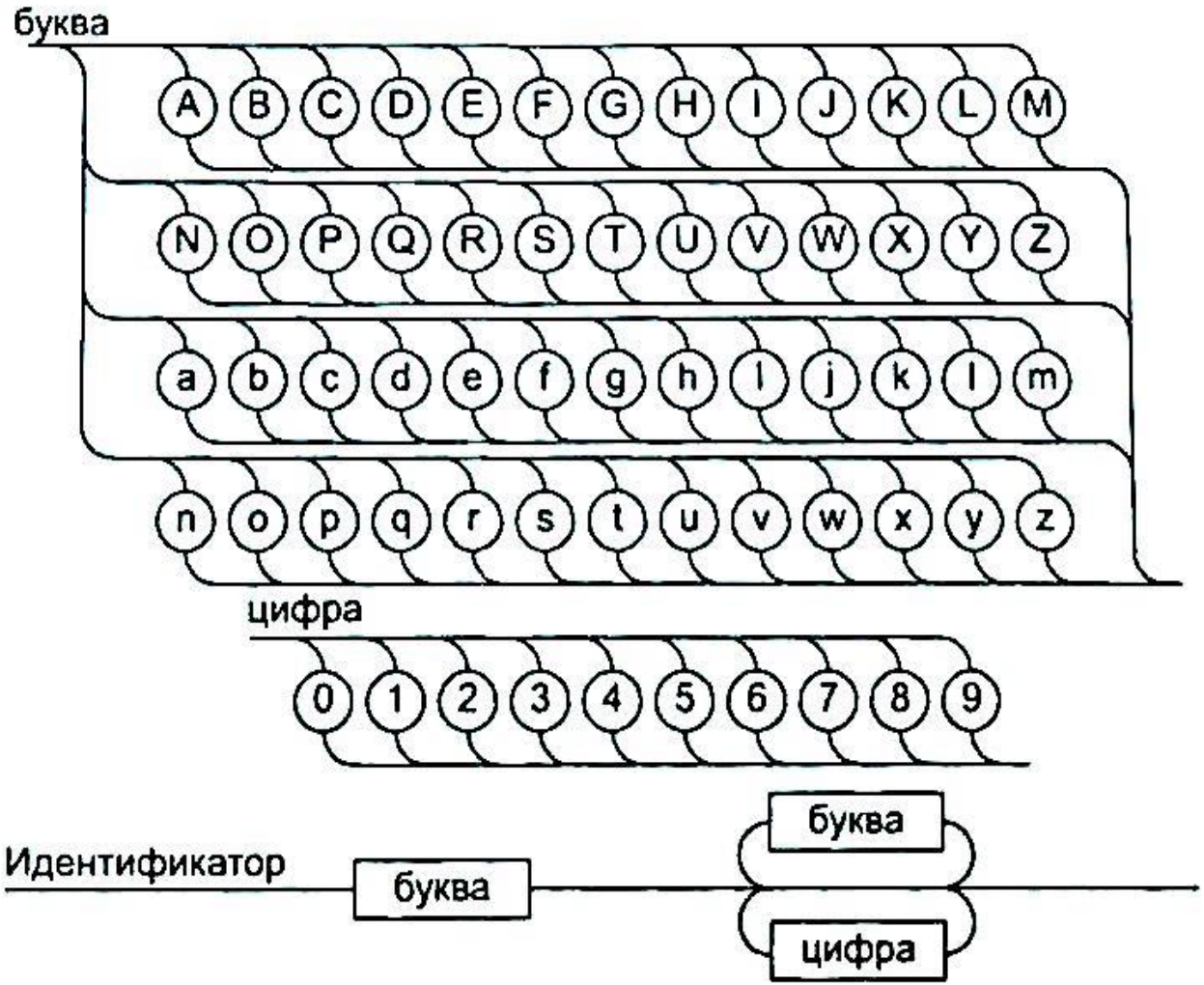


соединительные линии, обеспечивающие связь между терминальными и нетерминальными символами в правилах, заданных диаграммами Вирта



входная дуга с именем правила, определяющая его начало

Диаграмма Вирта



Пример описания идентификатора с использованием диаграмм Вирта

Представление языка с помощью формальной грамматики

Определение: *Формальной грамматикой* G называется четверка вида:

$$G = (N, T, P, S),$$

где N – конечное множество нетерминальных символов грамматики, нетерминальный алфавит; будем также использовать понятие «вспомогательный алфавит»; T – конечное множество терминальных символов грамматики, основной алфавит, алфавит определяемого языка, $N \cap T = \varnothing$; P – множество правил вывода грамматики, являющееся конечным подмножеством множества $(N \cup T)^{+} \times (N \cup T)^{*}$; элемент (α, β) множества P называется *правилом вывода* или *правилом подстановки* и записывается в виде $\alpha \rightarrow \beta$ (читается: «из цепочки α выводится цепочка β»); S – начальный символ грамматики, $S \in N$.

начальный символ *S* формальной грамматики

- $G = (N, T, P, S)$.
- Вывод предложений формального языка всегда начинается с начального символа грамматики.
- Это означает, что множество правил *P* грамматики обязательно должно включать правило вида $S \rightarrow \alpha$, где *S* – начальный символ грамматики.

Пример грамматики

Рассмотрим пример грамматики $G_{2.1}$:

$$G = (\{S, A\}, \{0, 1\}, \{S \to 0A1, 0A \to 00A1, A \to \varepsilon\}, S),$$

где $\{S, A\}$ – нетерминальный, вспомогательный алфавит; $\{0, 1\}$ – терминальный алфавит, алфавит определяемого языка; $\{S \to 0A1, 0A \to 00A1, A \to \varepsilon\}$ – множество правил вывода; S – начальный символ грамматики.

С помощью грамматики $G_{2.1}$ можно построить вывод предложения:

$$S \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000111,$$

где двойной стрелкой $\Rightarrow$ обозначена операция вывода с помощью правила грамматики.

Данная грамматика определяет язык $L = \{0^n1^n / n > 0\}$.

Правила записи

Пусть $G = (N, T, P, S)$ – грамматика. Для сокращения записи ее и введем ряд договоренностей:

1) для записи символов нетерминального алфавита N будем использовать заглавные буквы;

2) для записи символов терминального алфавита T будем использовать строчные буквы, цифры, специальные знаки +, *, (, ? и т.д.;

3) правило с начальным символом грамматики S будем записывать всегда первым в последовательности правил вывода;

4) если среди правил вывода имеется несколько правил с одинаковой левой частью, например, $U \to 0W$, $U \to 1V$, то группу таких правил будем записывать сокращенно, однократно выписывая строку из левой части правила, а различные правые части будем перечислять, разделяя символом | (или). Для приведенного примера правила запишутся так: $U \to 0W|0V$. Заметим, что здесь записаны два правила.

Примеры грамматик:

а) $S \to \varepsilon \mid 0S1$;

б) $S \to \varepsilon \mid (S) \mid SS$.

Выводимая цепочка

Определим понятие *выводимой цепочки*:

• S (начальный символ грамматики) – выводимая цепочка;

• если $\alpha\beta\gamma$ – выводимая цепочка и в грамматике имеется правило:

$$\beta \to \delta,$$

то $\alpha\delta\gamma$ – выводимая цепочка.

Обозначим операцию выводимости знаком $\Rightarrow$. Тогда понятие выводимой цепочки можно записать следующим образом: если $\alpha\beta\gamma$ – выводимая цепочка, в грамматике имеется правило $\beta \to \delta$ и имеет место $\alpha\beta\gamma \Rightarrow \alpha\delta\gamma$, то $\alpha\delta\gamma$ – выводимая цепочка.

Сентенциальная форма

Определение. Сентенциальной формой будем называть начальный символ грамматики и любую выводимую из начального символа цепочку. Для обозначения сентенциальной формы будем использовать операцию выводимости, дополненную символами * или +, т.е. $\Rightarrow^*$ или $\Rightarrow +$. Операция $\Rightarrow^*$ кратко обозначает вывод, полученный за ноль или более шагов вывода. Соответственно операция $\Rightarrow +$ обозначает вывод за один и более шагов. Рассмотрим пример. Пусть дана грамматика $S \rightarrow \varepsilon|(S)|S$. Тогда формула $S \Rightarrow^* \alpha$ обозначает множество выводимых из начального символа за ноль или более шагов грамматики цепочек, т.е. $\alpha = \{S, \varepsilon, (S), (), SS, (S)\,S,$

$() (), \ldots\}$

Применение правил ФГ

Подчеркнем, что формальная грамматика не является алгоритмом: правила подстановки – это не последовательность предписаний, а совокупность разрешений. Это означает: во-первых, правило вида $A \to \alpha$ понимается в грамматике как «A можно заменить на α» (а можно и не заменять), тогда как в алгоритме $A \to \alpha$ означало бы «A следует заменить на α» (нельзя не заменить); во-вторых, порядок применения правил в грамматике произволен, тогда как в алгоритме жестко задается порядок применения отдельных инструкций. Однако необходимо различать «разрешение применить правило» и «возможность применить правило», так как правило применяется только к такой цепочке, которая содержит вхождение левой части правила. Так, правило $AB \to CD$ можно применить к цепочке $AAEABC$, но нельзя применить к цепочке $AAEEBC$. Поэтому фактически порядок применения правил может диктоваться ими самими. Никаких внешних ограничений на порядок применения правил не существует.

Стратегии вывода

Стратегии вывода и редукции предложений языка. Существующие алгоритмы распознавания требуют определенной дисциплины вывода цепочек: вывод бывает лево- или правосторонний.

Правосторонний вывод получается, когда подстановка осуществляется всегда в самый правый нетерминальный символ.

Правосторонний вывод

<u>Пример.</u> Дана грамматика $G_{2.2}$:

$S \to AB$

$A \to CaC$

$B \to CbC$

$C \to c.$

Тогда для нее можно построить такой правосторонний вывод:

$$\underline{S} \Rightarrow A\underline{B} \Rightarrow ACb\underline{C} \Rightarrow A\underline{C}bc \Rightarrow \underline{A}cbc \Rightarrow Ca\underline{C}cbc \Rightarrow$$
$$\Rightarrow \underline{C}accbc \Rightarrow caccbc.$$

Левосторонний вывод: подстановка осуществляется в самый левый нетерминальный символ

<u>Пример.</u> Дана грамматика $G_{2.2}$:

$S \to AB$

$A \to CaC$

$B \to CbC$

$C \to c.$

$$\underline{S} \Rightarrow \underline{A}B \Rightarrow \underline{C}acB \Rightarrow cac\underline{B} \Rightarrow cac\underline{C}bC \Rightarrow caccb\underline{C} \Rightarrow caccbc.$$

Классификация формальных грамматик и автоматов по Хомскому

Пусть дана грамматика:

$$G = (N, T, P, S).$$

Обозначим через $V = T \cup N$ объединенный алфавит грамматики. Напомним, V^* – множество, содержащее все цепочки в алфавите V, включая пустую цепочку ε, а V^+ – множество, содержащее все цепочки в алфавите V, исключая пустую цепочку ε.

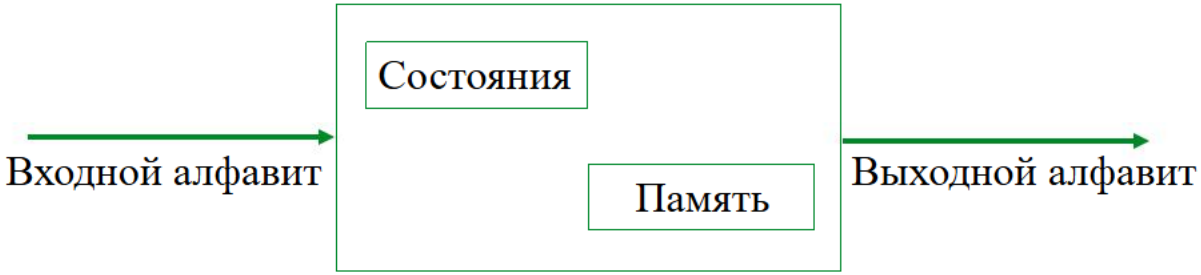
В основу классификации языков, грамматик автоматов по Хомскому положены виды ограничений, налагаемых на правила вывода грамматики.

Классификация ФГ по Хомскому

- В классификации Хомского классы называются типами. Каждый тип включает вид языка, его грамматики и распознающий автомат, обладающие определенным набором свойств.

Абстрактные автоматы

Каждому типу грамматики по Хомскому ставится в соответствие свой автомат:



Автомат характеризуется входным алфавитом, набором внутренних состояний и опционально выходным алфавитом и памятью.

Активация Wind
Чтобы активировать \

Абстрактный автомат (Abstract machine) – абстрактная модель устройства с одним входом, одним выходом и в каждый момент времени находящегося в одном состоянии из множества возможных. На вход абстрактного автомата поступают символы одного алфавита, на выходе появляются символы в общем случае другого алфавита.

Абстрактные автоматы

Абстрактный автомат определяется через кортеж:
$A=\{S, X, Y, \delta, \lambda\}$, где
S – множество состояний автомата,
X и Y – входной и выходной алфавиты соответственно
δ и λ – функции переходов и функции выхода:
$\delta: S \times X \rightarrow S$
$\lambda: S \times X \rightarrow Y$

Тип 0. Языки, грамматики с фразовой структурой, машина Тьюринга

Грамматика $G = (N,T,P,S)$ называется грамматикой типа 0 или грамматикой с фразовой структурой или грамматикой общего вида, если на ее правила вывода не наложено никаких ограничений, кроме единственного, учтенного в определении грамматики: из пустой цепочки ничего не выводится.

Правило грамматики имеет вид $\alpha \to \beta$, где $\alpha \in V^+$, $\beta \in V^*$.

Термин «фразовая структура» означает тот факт, что само предложение и рассматриваемые составляющие предложения представляют собой выводимую фразу,

Тип 0. Языки, грамматики с фразовой структурой

- Грамматика фразовой структуры (PSG), предложенная Ноамом Хомским, — это **формальная система правил**, описывающая синтаксис языка через иерархическое разбиение предложений на составляющие (фразы).
- Правила определяют, как нетерминальные символы (существительное, глагол) преобразуются в терминальные (конкретные слова), формируя основу для генеративного подхода к языку.
- Пример:
  Предложение (S)→ Именная группа (NP) + Глагольная группа (VP).

Пример

<u>Пример.</u> Язык $L = \{ww \mid w \in \{a, b\}^*\}$ может быть задан грамматикой типа 0 с правилами:

$$
\begin{array}{lll}
S \to CD & & \\
C \to aCA & Aa \to aA & C \to \varepsilon \\
C \to bCB & Ab \to bA & D \to \varepsilon \\
AD \to aD & Ba \to aB & \\
BD \to bD & Bb \to Bb. &
\end{array}
$$

Приведем вывод цепочки $abab$; подстроки, которые используются правилами подстановки, подчеркнуты:

$$\underline{S} \Rightarrow \underline{C}D \Rightarrow aC\underline{AD} \Rightarrow a\underline{C}aD \Rightarrow abC\underline{Ba}D \Rightarrow abCa\underline{BD} \Rightarrow ab\underline{C}abD \Rightarrow$$
$$\Rightarrow abab\underline{D} \Rightarrow abab.$$

Машина Тьюринга

Это самый общий тип грамматик.

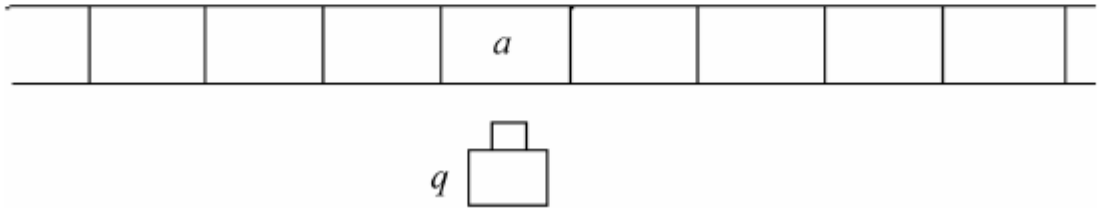
Практического применения грамматики, относящиеся только к типу 0, не имеют ввиду их сложности.

Грамматика общего вида определяет язык общего вида.

Задача анализа (редукции) языка общего вида решается автоматическим устройством, получившим название машина Тьюринга. Машина Тьюринга способна имитировать всех исполнителей, реализующих процесс пошагового вычисления, в котором каждый шаг вычисления достаточно элементарен. Другими словами, всякий интуитивный алгоритм может быть реализован с помощью некоторой машины Тьюринга.

Машина Тьюринга

В состав машины Тьюринга входит неограниченная в обе стороны лента, разделенная на ячейки, и управляющее устройство, также называемое головкой записи-чтения, способное находиться в одном из множества состояний. Число возможных состояний управляющего устройства конечно и точно задано.



Управляющее устройство может перемещаться влево и вправо по ленте, читать и записывать в ячейки символы некоторого конечного алфавита. Выделяется особый *пустой* символ, заполняющий все клетки ленты, кроме тех из них (конечного числа), на которых записаны входные данные.

Машина Тьюринга

Управляющее устройство работает согласно правилам перехода, которые представляют алгоритм, реализуемый данной машиной Тьюринга. Каждое правило перехода предписывает машине, в зависимости от текущего состояния и наблюдаемого в текущей клетке символа, записать в эту клетку новый символ, перейти в новое состояние и переместиться на одну клетку влево или вправо. Некоторые состояния машины Тьюринга могут быть помечены как *терминальные* (*финальные*), и переход в любое из них означает конец работы, остановку алгоритма.

Машина Тьюринга

Конкретная машина Тьюринга задается перечислением элементов множества символов алфавита A, множества состояний Q и набором правил, команд, по которым работает машина. Команда имеет вид: $qa \rightarrow q_1 a_1 d$. Если головка находится в состоянии q, а в обозреваемой ячейке записана буква a, то головка переходит в состояние q_1, в ячейку вместо a записывается a_1, головка делает движение d, которое имеет исполнения: на ячейку влево (L), на ячейку вправо (R), остаться на месте (N).

Автомат заканчивает работу в двух случаях:

1) пришел в финальное состояние, работа автомата завершается – задача успешно решена;

2) пришел в обычное состояние, отсутствует команда, которая может быть выполнена, работа автомата завершается – задача не имеет решения.

Возможен также случай бесконечной работы автомата – решение не может быть получено.

Рассмотрим пример машины Тьюринга для умножения чисел в единичной системе счисления.

Структура входной ленты имеет вид *ПППП × ВВВ = РРРР*, где ПППП – множимое; ВВВ – множитель; РРРР – результат, при этом множимое, множитель и результат строятся в алфавите {1}.

Алгоритм умножения следующий: помечаем первую единицу множителя ВВВ, заменяя ее на букву «а», и переносим единицу в конец результата РРРР; таким образом переносим все множимое ПППП за знак равенства. Затем заменяем все символы «а» множимого ПППП на единицы. Действие повторяется со второй, третьей и т.д. единицами множителя ВВВ.

Таким образом, умножение выполняется как ПППП + ПППП + + ПППП + и т.д. В сумме участвуют ВВВ слагаемых. Заметим, операция выполняется над цепочками символов, поэтому символом + обозначена операция конкатенации.

МТ: умножение чисел в единичной системе счисления

Таблица 2.3

№ п/п	Состояние	Входная лента	№ п/п	Состояние	Входная лента
1	q_0	$*111 \times 11 = *$	28	q_4	$*a\underline{a}a \times a1 = 11*$
2	q_0	…	29	q_4	…
3	q_0	$*111 \times 11 = *$	30	q_4	$*aaa \times a1 = 11*$
4	q_1	$*111 \times 11 = *$	31	q_5	$*aaa \times a1 = 111_$
5	q_2	$*111 \times a1 = *$	32	q_2	$*aaa \times a1 = 111*$
6	q_2	$*111 \times \underline{a}1 = *$	33	q_2	…
7	q_2	$*111 \times a1 = *$	34	q_2	$*aaa \times a1 = 111*$
8	q_3	$*111 \times a1 = *$	35	q_3	$*a\underline{a}a \times a1 = 111*$
9	q_4	$*11a \times a1 = *$	36	q_3	…
10	q_4	…	37	q_3	$*aaa \times a1 = 111*$
11	q_4	$*11a \times a1 = *$	38	q_6	$*\underline{a}aa \times a1 = 111*$
12	q_5	$*11a \times a1 = 1_$	39	q_6	…
13	q_2	$*11a \times a1 = 1*$	40	q_6	$*111 \times a1 = 111*$
14	q_2	…	41	q_7	$*111 \times \underline{a}1 = 111*$
15	q_2	$*11a \times a1 = 1*$	42	q_7	$*111 \times a1 = 111*$
16	q_3	$*11\underline{a} \times a1 = 1*$	43	q_2	$*111 \times aa = 111*$
17	q_3	$*11a \times a1 = 1*$	44	q_2	…
18	q_4	$*1a\underline{a} \times a1 = 1*$	45	q_2	$*111 \times aa = 111*$
19	q_4	…	46	q_3	$*111 \times aa = 111*$
20	q_4	$*1aa \times a1 = 1*$	47	q_4	$*11a \times aa = 111*$
21	q_5	$*1aa \times a1 = 11_$	48	q_4	
22	q_2	$*1aa \times a1 = 11*$	49	q_4	
23	q_2	…	50	q_5	
24	q_2	$*1aa \times a1 = 11*$	51	q_2	
25	q_3	$*1a\underline{a} \times a1 = 11*$	52	q_2	
26	q_3	$*1\underline{a}a \times a1 = 11*$	53	q_2	
27	q_3	$*1aa \times a1 = 11*$	54	q_3	

Подготовка	
q_0	Начальное состояние. Идем вправо до ×. Переходим в состояние q_1
q_1	Идем вправо до 1. Заменяем 1 на «а». Переходим в состояние q_2
Переносим все «1» из первого числа в результат	
q_2	Идем влево до ×. Переходим в состояние q_3
q_3	Идем влево до 1. Если 1 найден, заменяем 1 на «а» и переходим в состояние q_4 иначе идем вправо до * и переходим в состояние q_6
q_4	Идем вправо до *. Заменяем * на 1 со сдвигом вправо. Переходим в состояние q_5
q_5	Восстанавливаем * в конце строки. Переходим в состояние q_2
Обрабатываем каждый разряд второго числа	
q_6	Идем влево до ×, заменяя «а» на 1. Переходим в состояние q_7
q_7	Идем вправо до × или до =. Если найден 1, то заменяем 1 на «а», переходим в состояние q_2. Если найден =, то переходим в состояние q_8
Завершение	
q_8	Идем влево до ×, заменяя «а» на 1. Переходим в состояние q_9
q_9	Стоп. Успешное завершение

МТ: умножение чисел в единичной системе счисления

Таблица 2.3

№ п/п	Состояние	Входная лента	№ п/п	Состояние	Входная лента
1	q_0	$*111 \times 11 = *$	28	q_4	$*a\underline{a}a \times a1 = 11*$
2	q_0	…	29	q_4	…
3	q_0	$*111 \times 11 = *$	30	q_4	$*aaa \times a1 = 11*$
4	q_1	$*111 \times 11 = *$	31	q_5	$*aaa \times a1 = 111_$
5	q_2	$*111 \times a1 = *$	32	q_2	$*aaa \times a1 = 111*$
6	q_2	$*111 \times \underline{a}1 = *$	33	q_2	…
7	q_2	$*111 \times a1 = *$	34	q_2	$*aaa \times a1 = 111*$
8	q_3	$*111 \times a1 = *$	35	q_3	$*aa\underline{a} \times a1 = 111*$
9	q_4	$*11a \times a1 = *$	36	q_3	…
10	q_4	…	37	q_3	$*aaa \times a1 = 111*$
11	q_4	$*11a \times a1 = *$	38	q_6	$*\underline{a}aa \times a1 = 111*$
12	q_5	$*11a \times a1 = 1_$	39	q_6	…
13	q_2	$*11a \times a1 = 1*$	40	q_6	$*111 \times a1 = 111*$
14	q_2	…	41	q_7	$*111 \times \underline{a}1 = 111*$
15	q_2	$*11a \times a1 = 1*$	42	q_7	$*111 \times a1 = 111*$
16	q_3	$*11\underline{a} \times a1 = 1*$	43	q_2	$*111 \times aa = 111*$
17	q_3	$*11a \times a1 = 1*$	44	q_2	…
18	q_4	$*1a\underline{a} \times a1 = 1*$	45	q_2	$*111 \times aa = 111*$
19	q_4	…	46	q_3	$*111 \times aa = 111*$
20	q_4	$*1aa \times a1 = 1*$	47	q_4	$*11a \times aa = 111*$
21	q_5	$*1aa \times a1 = 11_$	48	q_4	…
22	q_2	$*1aa \times a1 = 11*$	49	q_4	$*11a \times aa = 111*$
23	q_2	…	50	q_5	$*11a \times aa = 1111_$
24	q_2	$*1aa \times a1 = 11*$	51	q_2	$*11a \times aa = 1111*$
25	q_3	$*1a\underline{a} \times a1 = 11*$	52	q_2	…
26	q_3	$*1\underline{a}a \times a1 = 11*$	53	q_2	$*11a \times aa = 1111*$
27	q_3	$*1aa \times a1 = 11*$	54	q_3	$*11\underline{a} \times aa = 1111*$

Рассмотрим пример машины Тьюринга для умножения чисел в единичной системе счисления.

Окончание табл. 2.3

№ п/п	Состояние	Входная лента	№ п/п	Состояние	Входная лента
55	q_3	$*11a \times aa = 1111*$	71	q_2	…
56	q_4	$*1a\underline{a} \times aa = 1111*$	72	q_2	$*\underline{a}aa \times aa = 111111*$
57	q_4	…	73	q_3	$*\underline{a}a\underline{a} \times aa = 111111*$
58	q_4	$*1aa \times aa = 1111*$	74	q_3	…
59	q_5	$*1aa \times aa = 11111_$	75	q_3	$*\underline{a}aa \times aa = 111111*$
60	q_2	$*1aa \times aa = 11111*$	76	q_6	$*\underline{a}aa \times aa = 111111*$
61	q_2	…	77	q_6	$*1\underline{a}a \times aa = 111111*$
62	q_2	$*1aa \times aa = 11111*$	78	q_6	$*11\underline{a} \times aa = 111111*$
63	q_3	$*1a\underline{a} \times aa = 11111*$	79	q_6	$*111 \times aa = 111111*$
64	q_3	$*1\underline{a}a \times aa = 11111*$	80	q_7	$*111 \times \underline{a}a = 111111*$
65	q_3	$*1aa \times aa = 11111*$	81	q_7	$*111 \times a\underline{a} = 111111*$
66	q_4	$*a\underline{a}a \times aa = 11111*$	82	q_7	$*111 \times aa = 111111*$
67	q_4	…	83	q_8	$*111 \times a\underline{a} = 111111*$
68	q_4	$*aa\underline{a} \times aa = 11111*$	84	q_8	$*111 \times \underline{a}1 = 111111*$
69	q_5	$*\underline{a}aa \times aa = 111111_$	85	q_8	$*111 \times 11 = 111111*$
70	q_2	$*\underline{a}aa \times aa = 111111*$	86	q_9	$*111 \times 11 = 111111*$
			Стоп		

Рассмотрим пример машины Тьюринга для умножения чисел в единичной системе счисления.

Система команд машины Тьюринга для умножения единичных чисел приведена в табл. 2.1.

Таблица 2.1

q_0	$q_0 1 \to q_0 1 R$	$q_0 * \to q_0 * R$	$q_0 \times \to q_1 \times R$		
q_1	$q_1 1 \to q_2 a R$				
q_2	$q_2 1 \to q_2 1 L$	$q_2 = \to q_2 = L$	$q_2 a \to q_2 a L$	$q_2 \times \to q_3 \times L$	
q_3	$q_3 a \to q_3 a L$	$q_3 1 \to q_4 a R$	$q_3 * \to q_6 * R$		
q_4	$q_4 1 \to q_4 1 R$	$q_4 \times \to q_4 \times R$	$q_4 = \to q_4 = R$	$q_4 a \to q_4 a R$	$q_4 * \to q_5 1 R$
q_5	$q_5 \to q_2 * L$				
q_6	$q_6 a \to q_6 1 R$	$q_6 \times \to q_7 \times R$			
q_7	$q_7 1 \to q_2 a R$	$q_7 a \to q_7 a R$	$q_7 = \to q_8 = L$		
q_8	$q_8 a \to q_8 1 L$	$q_8 \times \to q_9 \times N$			
q_8	Стоп				

Рассмотрим пример машины Тьюринга для умножения чисел в единичной системе счисления.

Описание состояний машины Тьюринга из табл. 2.1 приведено в табл. 2.2.

Таблица 2.2

Подготовка	
q_0	Начальное состояние. Идем вправо до ×. Переходим в состояние q_1
q_1	Идем вправо до 1. Заменяем 1 на «а». Переходим в состояние q_2
Переносим все «1» из первого числа в результат	
q_2	Идем влево до ×. Переходим в состояние q_3
q_3	Идем влево до 1. Если 1 найден, заменяем 1 на «а» и переходим в состояние q_4 иначе идем вправо до * и переходим в состояние q_6
q_4	Идем вправо до *. Заменяем * на 1 со сдвигом вправо. Переходим в состояние q_5
q_5	Восстанавливаем * в конце строки. Переходим в состояние q_2
Обрабатываем каждый разряд второго числа	
q_6	Идем влево до ×, заменяя «а» на 1. Переходим в состояние q_7
q_7	Идем вправо до × или до =. Если найден 1, то заменяем 1 на «а», переходим в состояние q_2. Если найден =, то переходим в состояние q_8
Завершение	
q_8	Идем влево до ×, заменяя «а» на 1. Переходим в состояние q_9
q_9	Стоп. Успешное завершение

Алгоритмическая разрешимость

С машиной Тьюринга связано понятие алгоритмической разрешимости и алгоритмической неразрешимости проблемы. Под проблемой здесь понимается многократно решаемая вычислительная задача.

Алгоритмически разрешимая проблема – это та задача, которая решается за конечное число шагов на машине Тьюринга. Например, задача извлечения квадратного корня из целого числа алгоритмически разрешима.

Алгоритмическая разрешимость

Алгоритмически неразрешимая проблема представляет собой задачу, которая не всегда решается за конечное число шагов на машине Тьюринга.

Например (десятая проблема Гильберта): пусть задан многочлен P n-й степени с целыми коэффициентами. Существует ли алгоритм, который находит решение уравнения $P = 0$ в целых числах? Ответ отрицательный.

Языки с фразовой структурой – это самые сложные языки, которые могут быть заданы только грамматикой, относящейся к типу 0. Если язык относится к типу 0, то для него невозможно построить компилятор, который гарантированно выполнял бы разбор предложений языка за ограниченное время на основе ограниченных вычислительных ресурсов.

Тип 1. Контекстно-зависимые языки, грамматики, линейно ограниченные автоматы

Грамматику типа 1 можно определить как *неукорачивающую*, либо как *контекстно-зависимую*.

Смысл неукорачивающей грамматики заключается в том, что на правила накладывается ограничение: каждое правило вывода $\alpha \to \beta$ грамматики должно отвечать ограничению $|\alpha| \leq |\beta|$. Это означает, что выводимая с помощью правила цепочка не должна становиться короче исходной цепочки.

Правила вывода контекстно-зависимой грамматики

$$\alpha B\gamma \to \alpha\beta\gamma,$$

где B – нетерминальный символ; α, γ – произвольные цепочки из алфавита V^*; β – произвольные цепочки из алфавита V^+. Термин «контекстно-зависимые» отражает тот факт, что нетерминальный символ B можно заменить на цепочку β лишь в контексте α, γ.

В правиле $\alpha B\gamma \to \alpha\beta\gamma$ контекстно-зависимой грамматики α часто называют левым контекстом, а γ – правым контекстом. Обратим внимание, что в правиле вывода $\alpha \in V^*$ и $\gamma \in V^*$. Это означает, что один из контекстов может быть пустым. Если пустыми являются оба контекста, то грамматика, оставаясь контекстно-зависимой, относится к типу КС-грамматик.

Пример 1. К3 грамматика

<u>Пример</u>: язык $L = \{a^n\ (b\ c)^n \mid n \geq 1\}$ может быть порожден грамматикой следующего вида:

$S \to aSBC \mid abc$

$cB \to cb$

$bC \to bc.$

Приведем вывод предложения $aabcbc$:

$$\underline{S} \Rightarrow a\underline{S}BC \Rightarrow aab\underline{cB}C \Rightarrow aabc\underline{bC} \Rightarrow aabcbc.$$

Пример 2

Контекстно-зависимый язык

$L = \{a^n b^n c^n \mid n > 0\}$ задается грамматикой с правилами:

$S \to ABC$

$B \to b$

$bD \to bb$

$EC \to DC$

$A \to a$

$CD \to ED$

$B \to ABCD$

$C \to c$

$ED \to EC.$

Грамматики типа 1

- Грамматики типа 1 и все последующие часто называют грамматиками непосредственных составляющих.
- Они обладают очень важным в лингвистическом аспекте свойством. Терминальные символы понимаются как слова некоторого языка, нетерминальные символы – как синтаксические категории (например, глагол, существительное в естественном языке; оператор, выражение в языке программирования), начальный символ – как «предложение», а выводимые терминальные цепочки – как правильные предложения данного языка.
- Тогда вывод предложения естественно интерпретируется как его синтаксическая структура, представленная в терминах непосредственно составляющих, т.е. известным в лингвистике способом.

Линейно ограниченный автомат

Определение:

Линейно ограниченный автомат (англ. *linear bounded automata*, *lba*) — недетерминированная одноленточная машина Тьюринга, которая никогда не покидает те ячейки, на которых размещен ее ввод.

Более формально:

Определение:

Линейно ограниченный автомат — формальная система $M = \langle Q, \Sigma, \Gamma, \delta, q_0, F \rangle$, в которой

- Q — множество состояний,
- $q_0 \in Q$ — начальное состояние,
- $F \subset Q$ — множество конечных состояний,
- Γ — алфавит допустимых символов ленты,
- $\Sigma \subset \Gamma$ — алфавит входных символов, который содержит два особых символа: $\#$ и $\$$ — левый и правый маркеры, находящиеся с самого начала на концах входной цепочки для того, чтобы предотвращать выход головки ленты за пределы участка, на котором размещается входная цепочка (считается, что маркеры могут использоваться только в этой роли: на место маркера нельзя записать какой-нибудь другой символ ленты, и никакой символ ленты не может быть заменен каким-нибудь маркером),
- δ — отображение типа $Q \times \Gamma \to 2^{Q \times \Gamma \times \{\leftarrow, \rightarrow\}}$.

Активация Windows

Связь линейно ограниченных автоматов с контекстно-зависимыми языками

Теорема:

Если L — контекстно-зависимый язык, то язык L принимается некоторым линейно ограниченным автоматом.

Грамматики типа 1

- Язык, заданный КЗ-грамматикой распознается линейно-ограниченным автоматом, устроенный, как и машина Тьюринга, однако в процессе выполнения каждой команды линейно-ограниченного автомата длина цепочки, получаемой на входной ленте, не может стать длиннее исходной цепочки.
- Благодаря этому свойству задача распознавания всегда заканчивается за конечное число шагов.
- Все языки программирования являются контекстно-зависимыми языками. Однако из соображений эффективности контекстно- зависимые грамматики и линейно-ограниченные автоматы в трансляторах не используются.

Тип 2. Контекстно-свободные языки, грамматики, автоматы с магазинной памятью

Правила КС-грамматики имеют вид:

$$A \to \alpha,$$

где A – нетерминальный символ, $\alpha \in V^+$. Правило подстановки применяется независимо от контекста, в котором находится нетерминальный символ A.

<u>Пример.</u> Контекстно-свободный язык $L = \{a^n\, b^n \mid n > 0\}$ определяется КС-грамматикой с правилами:

$I \to a\, I\, b$

$I \to a\, b.$

Рассмотрим вывод цепочки $a\ a\ b\ b$:

$$I \Rightarrow aIb \Rightarrow aabb.$$

Пример 3. КС -грамматика

Тот же язык $L = \{a^n b^n c^n \mid n > 0\}$ может быть порожден грамматикой с правилами:

$S \to aSBC$ $\quad$ $S \to aBC$ $\quad$ $CB \to BC$

$aB \to ab$ $\quad$ $bB \to bb$

$bC \to bc$ $\quad$ $cC \to cc.$

КС-грамматики и автомат с магазинной памятью

Определение. *Распознающим автоматом* для контекстно-свободных языков является автомат с магазинной памятью (или стеком) (рис. 2.2).

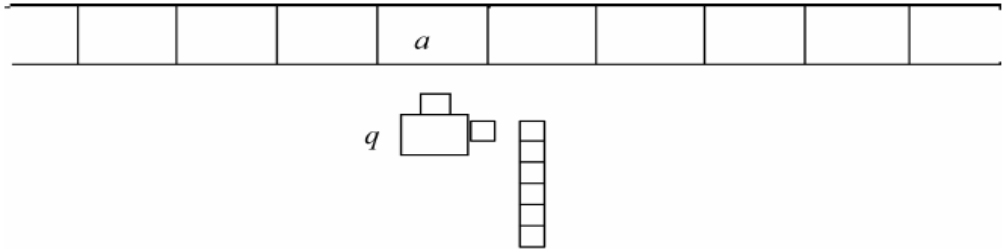


Рис. 2.2. Автомат с магазинной памятью

Конфигурация автомата определяется выражением (a, q, b). Команда автомата имеет вид $(q, a, b) \rightarrow (q_1, b_1, r)$, где a – символ входной ленты; q, q_1 – состояния автомата; b, b_1 – верхние символы магазина; r = {вправо, на месте} – продвижение по входной ленте.

Такт работы автомата следующий: если конфигурация автомата совпадает с левой частью одной из команд, то команда выполняется; выполнение команды заключается в замене верхнего символа b магазина на символ b_1, переходе из состояния q в состояние q_1, перемещении по входной ленте в соответствии со значением r.

КС-грамматики и автомат с магазинной памятью

Существенным свойством автомата с магазинной памятью является то, что направленная ко входной ленте головка предназначена только для чтения, при этом после чтения входного символа и выполнения некоторых действий с магазинной памятью в течение нескольких тактов головка чтения продвигается вперед по входной ленте. Такой принцип работы свидетельствует об эффективности автомата с магазинной памятью.

Пример

Пример. Рассмотрим задачу проверки корректности вложенности круглых скобок.

Грамматика может быть определена следующим образом:

1. $S \to (A)\ A$.

2. $A \to S$.

3. $A \to \varepsilon$.

Определим систему команд автомата следующим образом:

1. $q, (, \vdash \to q, A, R$
2. $q, (, A \to q, A, R$
3. $q,), A, \to q, \varepsilon, R$
4. $q, \$, \vdash \to$ Стоп.

Пример. Рассмотрим задачу проверки корректности вложенности круглых скобок

Определим систему команд автомата следующим образом:

1. $q, (, \vdash \rightarrow q, A, R$
2. $q, (, A \rightarrow q, A, R$
3. $q,), A, \rightarrow q, \varepsilon, R$
4. $q, \$, \vdash \rightarrow$ Стоп.

Из представленного описания видно, что поведение автомата имитирует метод распознавания с использованием счетчика, только вместо счетчика используется символ A, который записывается в стек и стирается из него, отражая разность между прочитанными открывающимися и закрывающимися скобками. Рассмотрим работу данного автомата с магазинной памятью на примере распознавания цепочки (() ()) $. Протокол работы автомата при распознавании цепочки (() ()) $ приведен в табл. 2.4.

Таблица 2.4

Номер шага	Содержимое стека	Состояние автомата	Остаток входной цепочки	Номер применяемого правила
1	$\vdash$	q	(() ()) $	1
2	$\vdash A$	q	() ()) $	2
3	$\vdash A\,A$	q	) ()) $	3
4	$\vdash A$	q	()) $	2
5	$\vdash A\,A$	q	)) $	3
6	$\vdash A$	q	) $	3
7	$\vdash$	q	$	Стоп